Python Loading and the Launcher

How the shared launcher starts each tool's Python file: the optimized run, the memory cap on the C allocator, the periodic memory trim, and how the tool file is loaded. It also covers the three opt-out flags.

LAUNCHER · -OO · MALLOC_ARENA_MAX · OPT-OUT FLAGS

SUMMARY · READ-ONLY REVIEW

One script, eleven copies

The launcher is a small Python script installed at `/usr/local/bin/asl_dvs_launch.py`. Each tool's systemd service starts Python through it, with the tool's file as the first argument. The launcher prepares the process, then runs the tool as if Python had started it directly.

The same launcher text is written by eleven tool files: the full and pi02w builds of the Dashboard and SysMon, instmon, 44helper, Node Talker, amp Node Talker, Node Recorder, wifimon and pibrary. The repo rules require the text to be identical in every copy, and a hash comparison of the extracted text confirmed that it is.

The first run

2.1 Re-running Python

If the launcher's marker variable (`ASL_DVS_LAUNCH`) is not set, it is the first run. It builds a list of extra flags and a copy of the environment, then:

1. **Optimization.** If Python is not already running at optimization level 2, and the `launch_no_optimize` flag is absent, it adds `-OO`.
2. **Allocator cap.** If `MALLOC_ARENA_MAX` is not already set, and the `launch_no_arena_cap` flag is absent, it sets `MALLOC_ARENA_MAX=2` and records the marker as `arena`.
3. **Re-run.** If either step asked for a change, it replaces its own process with `os.execve`, using the Python executable, the extra flags and the original arguments. If the replacement fails, it carries on in the current process.

`execve` replaces the program in the same process, so the process ID does not change. systemd, which runs the service as Type=notify, therefore sees the same process from start to finish.

2.2 In the second run

The second run sees the marker. It removes the marker from its environment. If the marker said `arena`, it also removes `MALLOC_ARENA_MAX`. The cap has already taken effect in the process by then, because the C allocator reads it at start. Removing it stops the cap being passed to any child process the tool starts later, so those children get the usual allocator behaviour.

A background thread calls malloc_trim

Unless the `launch_no_trim` flag is present, the launcher starts a daemon thread named `mem-trim`. The thread:

1. Sleeps 60 seconds, to let the tool start.
2. Loads the C library and gets its `malloc_trim` function. If the library or function is missing, the thread ends quietly.
3. Calls `malloc_trim(0)`, then sleeps 300 seconds, and repeats for as long as the process runs.

The call asks the allocator to return freed memory to the operating system. It does not free memory that the tool still uses. Because the thread is a daemon, it does not keep the process alive when the tool exits.

How the tool's Python is run

1. **Find the file.** The first argument is resolved with `realpath`, so a symbolic link is followed to the real file.
2. **Set the arguments.** `sys.argv` becomes the file's path followed by the rest of the original arguments, so the tool sees the same arguments it would see if run directly.
3. **Set the import path.** The file's folder is put first on `sys.path`, so modules beside the tool import correctly.
4. **Run as the main program.** If the file's base name is a valid Python identifier, the launcher runs it with `runpy.run_module` under the name `__main__`. Otherwise, such as a file name with a hyphen, it loads the file by path with `importlib`, registers it as `__main__`, and runs it.

The two paths give the same result for the tool. The second exists because a module name with a hyphen cannot be imported by name.

Why the optimized run is safe for these tools

Running Python with `-OO` removes `assert` statements, sets `__debug__` to false, and removes docstrings from the compiled code. The gain is a smaller compiled footprint and slightly less work at run time.

- **Asserts.** The pi02w tools use no `assert` statement that a run depends on. A check that matters is written as an `if` and an error, so removing asserts does not remove a check.
- **Docstrings.** Docstrings are not read by the tools' logic. The one that matters for the book is in a wrapper, so its removal does not change behaviour.
- **Debug checks.** No code path depends on `__debug__` being true.

This was checked by reading the pi02w tools, not by running the full test set on a node. That is the next thing to confirm.

What MALLOC_ARENA_MAX=2 does

glibc gives each thread its own memory arena to reduce lock contention. On a small device with many threads, the arenas can hold a lot of memory that is not in use. Setting `MALLOC_ARENA_MAX=2` limits the process to two arenas, which trades a little speed under heavy threading for a smaller footprint.

The launcher applies the cap to the tool process only. The removal in the second run (Part 2.2) stops the cap passing to child processes the tool starts.

Turning each optimization off

FLAG FILE	TURNS OFF	EFFECT
`/etc/asl_dvs/launch_no_optimize`	The `-OO` run	Python runs with its normal asserts and docstrings
`/etc/asl_dvs/launch_no_arena_cap`	The `MALLOC_ARENA_MAX=2` cap	The allocator uses its default arenas
`/etc/asl_dvs/launch_no_trim`	The periodic `malloc_trim` thread	Freed memory is not returned on a timer

The launcher checks for the flag files each time it starts. A change therefore takes effect when the service next starts, not while it is running. The flag files keep the `asl_dvs` system name, as the naming rules require.

The user manual, Appendix D, describes these flags and gives a table of the memory each tool uses with and without them. Those figures come from the manual and were not measured for this summary.

What the launcher shows

8.1 One launcher, one behaviour

Because every tool starts through the same script, a change to the launcher changes all eleven tools at once. The identical-text rule keeps that change predictable.

8.2 Running a file by hand skips the launcher

A tool run directly with `python3` does not get `-OO`, the cap or the trim thread. Only the service path uses the launcher, so a person testing a tool by hand sees different memory behaviour from the service.

8.3 Fallbacks

- If the re-run fails, the tool carries on in the first process, without the flags.
- If the trim library is missing, the trim thread ends and nothing else changes.

8.4 Not checked

- The memory figures in the manual, which were not measured here.
- Whether every tool's test set passes under `-OO` on a running node.
- Whether the arena cap changes speed for any tool under load.